

AMDP 練習 2：第一個 AMDP Method——Signature 規則與雙 Internal Table 輸出

Lecture

am01 已經帶出三個 AMDP 簽章限制 (參數須 **VALUE()**、**DEFAULT** 只能常數、**USING** 要列清楚依賴的資料庫物件)，這題把「Signature 規則」這個主題講完整，並且示範 AMDP 一個很重要、一般 ABAP Method 做不到的特性：一次呼叫，同時輸出兩個獨立的 **Internal Table**。

一般 ABAP Method 的 **RETURNING** 只能有一個回傳值，如果要回傳多筆不同形狀的結果 (例如「查詢明細」跟「這批明細的統計摘要」)，通常得拆成兩次 Method 呼叫，或者硬把兩種結果包進同一個巢狀結構。

AMDP Method 完全不支援 **RETURNING** / **CHANGING** (AMDP 只認 **IMPORTING** / **EXPORTING**)，但反過來，**EXPORTING** 可以宣告任意多個 **Table Type** 參數，SQLScript 程序本體可以在同一次執行裡，把好幾個不同形狀的查詢結果分別指派給不同的 **EXPORTING** 參數，一次呼叫、一次資料庫往返 (round trip)，就把「明細」跟「彙總統計」兩張表都算好、都傳回來。

這對應到 Code-to-Data 的核心價值：如果這個需求是「查詢明細 + 這批明細的統計」，Open SQL 的做法通常是查一次明細 (**SELECT ... INTO TABLE**)、在 ABAP 端再用 **LOOP/COLLECT** 或另一次 **SELECT ... AGGREGATE FUNCTIONS** 算統計——兩個查詢，或一次查詢 + 應用層迴圈運算。AMDP 可以把「查明細」跟「算統計」這兩件事都寫在同一個 SQLScript 程序裡，資料庫引擎各自查各自算，呼叫端一次呼叫就同時拿到兩個結果，不需要自己在應用層做第二次運算。

學習目標

- 完整掌握 AMDP Method 的簽章規則：只有 **IMPORTING** / **EXPORTING** (沒有 **RETURNING** / **CHANGING**)，所有參數都要 **VALUE(...)**，**DEFAULT** 只能常數
- 會宣告一個同時 **EXPORTING** 兩個 **Internal Table** 的 AMDP Method，並在 SQLScript 本體裡對兩個 **EXPORTING** 參數各自指派一個 **SELECT** 結果
- 理解這跟一般 ABAP Method 的 **RETURNING** (只能單一值) 在能力上的本質差異
- 呼叫端拿到兩張表之後，各自用 **cl_salv_table** (承 OOP op11) 呈現成 ALV，實際看到「一次 AMDP 呼叫、兩個獨立結果集、兩個 ALV 畫面」的效果

事前準備

ADT 端物件已由課程準備好 (**\$TMP**)：

- **ZCL_AM02_FLIGHT_STATS**——AMDP 類別，**get_flight_stats** 方法同時輸出 **SFLIGHT** 明細 (**et_flights**) 與依 **connid** 分組的統計摘要 (**et_stats**：航班筆數、平均/最低/最高票價)
- **ZR_AM02_DEMO**——demo 程式，帶一個 Selection Screen 參數 **p_carriid** (預設 **LH**)，呼叫 AMDP 後用 **cl_salv_table** 分別顯示兩張表

題目需求 (對照已建好的答案物件)

1. **ZCL_AM02_FLIGHT_STATS** 的簽章與 SQLScript 本體：

```
``abap CLASS zcl_am02_flight_stats DEFINITION PUBLIC FINAL CREATE PUBLIC. PUBLIC SECTION. INTERFACES  
if_amdp_marker_hdb.
```

```
TYPES: BEGIN OF ty_flight, carrid TYPE s_carr_id, connid TYPE s_conn_id, fldate TYPE s_date, price TYPE s_price,  
currency TYPE s_currcode, END OF ty_flight, tt_flight TYPE STANDARD TABLE OF ty_flight WITH EMPTY KEY,
```

```
BEGIN OF ty_stats, connid TYPE s_conn_id, flight_cnt TYPE i, avg_price TYPE s_price, min_price TYPE s_price,  
max_price TYPE s_price, END OF ty_stats, tt_stats TYPE STANDARD TABLE OF ty_stats WITH EMPTY KEY.
```

```
CLASS-METHODS get_flight_stats IMPORTING VALUE(iv_mandt) TYPE mandt VALUE(iv_carrid) TYPE s_carr_id  
EXPORTING VALUE(et_flights) TYPE tt_flight VALUE(et_stats) TYPE tt_stats. ENDCLASS.
```

```
CLASS zcl_am02_flight_stats IMPLEMENTATION.
```

```
METHOD get_flight_stats BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT OPTIONS READ-ONLY  
USING sflight.
```

```
et_flights = SELECT carrid, connid, fldate, price, currency FROM sflight WHERE mandt = :iv_mandt AND carrid  
= :iv_carrid ORDER BY connid, fldate;
```

```
et_stats = SELECT connid, COUNT(*) AS flight_cnt, AVG(price) AS avg_price, MIN(price) AS min_price,  
MAX(price) AS max_price FROM sflight WHERE mandt = :iv_mandt AND carrid = :iv_carrid GROUP BY connid  
ORDER BY connid;
```

```
ENDMETHOD.
```

```
ENDCLASS. ``
```

兩個重點：

- **et_flights / et_stats** 是兩個平行的 **EXPORTING** 參數，**SQLScript** 本體裡各自一句 **SELECT** 指派，不是把統計結果塞進明細表的某種巢狀欄位——兩張表形狀完全不同（**ty_flight** 5 欄 vs **ty_stats** 5 欄但語意不同），各自獨立
- 承 **am01** 的教訓，這題一樣手動 **WHERE mandt = :iv_mandt**——這套系統多 Client 灌了同一批示範資料，AMDP 不會像 Open SQL 自動處理這件事，兩個 **SELECT** 都要記得加

1. **ZR_AM02_DEMO** 呼叫端：

```
``abap REPORT zr_am02_demo.
```

```
PARAMETERS p_carrid TYPE s_carr_id DEFAULT 'LH'.
```

```
START-OF-SELECTION.
```

```
zcl_am02_flight_stats=>get_flight_stats( EXPORTING iv_mandt = sy-mandt iv_carrid = p_carrid IMPORTING  
et_flights = DATA(lt_flights) et_stats = DATA(lt_stats) ).
```

```
cl_salv_table=>factory( IMPORTING r_salv_table = DATA(lo_alv_flights) CHANGING t_table = lt_flights ).  
lo_alv_flights->get_columns()->set_optimize( abap_true ). lo_alv_flights->display( ).
```

```
cl_salv_table=>factory( IMPORTING r_salv_table = DATA(lo_alv_stats) CHANGING t_table = lt_stats ).  
lo_alv_stats->get_columns()->set_optimize( abap_true ). lo_alv_stats->display( ). ``
```

一次 AMDP 呼叫拿到 `lt_flights`、`lt_stats` 兩個獨立的 Internal Table 之後，各自呼叫一次 `cl_salv_table=>factory( ) → display( )`——第一個 `display( )` 會先顯示明細 ALV，使用者按「返回」(F3) 離開後，程式才會繼續往下跑，顯示第二個統計 ALV，這是 `cl_salv_table` 的標準行為 (`display( )` 是全螢幕、會 block 住往下執行，直到使用者離開這個畫面)，不是程式有問題卡住。

預期輸出

以預設值 `p_carrid = 'LH'` 執行：

第一個 ALV (明細，`et_flights`)：SFLIGHT 裡 `CARRID = 'LH'` 的所有航班，依 `CONNID/FLDATE` 排序，欄位為 `CARRID/CONNID/FLDATE/PRICE/CURRENCY`。

第二個 ALV (統計，`et_stats`)：依 `CONNID` 分組的摘要，例如 (實測數字，這套系統的 SFLIGHT 資料)：

CONNID	FLIGHT_CNT	AVG_PRICE	MIN_PRICE	MAX_PRICE
0400	16	687.74	666.00	999.99
0401	15	666.00	666.00	666.00
0402	15	666.00	666.00	666.00
2402	15	242.00	242.00	242.00
2407	15	242.00	242.00	242.00

(0400 平均票價偏高，是因為裡面有一筆 999.99 的異常票價把平均拉高——這正好可以用來對照明細表，驗證統計數字沒有算錯。)

團隊實務備註

- 這題的兩張表是各自獨立的 **SELECT**，不是「一次查詢同時算出兩種形狀的結果」：SQLScript 程序本體裡本來就可以照順序寫任意多個語句 (本題是兩句 **SELECT**)，每句指派給哪個 **EXPORTING** 變數，就是那個變數的最終結果；不需要、也沒有辦法用「一句 SQL 神奇地生出兩種不同形狀的表格」
- AMDP 沒有 **RETURNING/CHANGING** 這件事，寫慣一般 **ABAP Method** 的人很容易手滑：如果你在 AMDP 方法簽章寫 **RETURNING VALUE(rt_result) TYPE ...**，語法檢查會直接擋下來 (AMDP 只認 **IMPORTING/EXPORTING**)，這不是本題才有的限制，是 AMDP 语言规范本身的限制
- `cl_salv_table->display( )` 這種會產生全螢幕畫面的呼叫，沒辦法透過 **ADT** 的無頭 **programrun API 自動驗證**——這題開發時先臨時把 ALV 呼叫換成 **WRITE** 迴圈確認 AMDP 回傳的兩張表資料正確 (明細筆數、統計數字都對得上)，驗證過資料邏輯無誤後，才換回正式的 ALV 版本；ALV 畫面本身的呈現效果，仍然要你在 SAP GUI 執行 **ZR_AM02_DEMO** (F8) 親眼確認兩個 ALV 畫面都正常跳出
- `get_columns( )->set_optimize( abap_true )` 只是讓 ALV 欄寬自動依內容調整，跟這題的 AMDP 主題沒有直接關係，純粹是 op11 教過的 `cl_salv_table` 慣用手法，讓畫面好看一點

思考題

- 如果把 `et_stats` 的 SQLScript 改成先算 `et_flights`、再用 **SELECT ... FROM :et_flights** (對剛剛查出來的明細表變數再做一次聚合，而不是重新 **SELECT ... FROM sflight**) 理論上可不可行？這樣做對「資料庫要重新掃描 `sflight` 兩次」這件事有沒有幫助？ (提示：SQLScript 允許把一個 Table Variable 當作後續查詢的資料來源)

2. 如果呼叫端只想要統計摘要、不需要明細，目前的簽章設計只能兩個都拿或都不拿（因為都是 `EXPORTING`，呼叫端可以只接其中一個變數，但資料庫端還是兩句 `SELECT` 都會執行）——這樣有沒有效能上的浪費？如果真的常常只需要其中一個，AMDP 方法設計上可以怎麼調整？
3. 這題的 `ty_stats` 用 `flight_cnt TYPE i` 承接 SQLScript 的 `COUNT(*)`——如果改成更精確的型別（例如 `int4`），啟用/執行結果會不會不一樣？（提示：可以動手試試看，比較 HANA SQLScript 聚合函數的結果型別跟 ABAP 端宣告型別之間，系統是怎麼做自動轉換的）

答案

見 `zcl_am02_flight_stats.clas.abap`、`zr_am02_demo.prog.abap`（SAP 端物件 `ZCL_AM02_FLIGHT_STATS` / `ZR_AM02_DEMO`，套件 `$TMP`）。ALV 畫面效果請在 SAP GUI 執行 `ZR_AM02_DEMO`（F8）實際確認。